

I2C OLED Display Experiment

I2C OLED Display Experiment

- 1. I2C OLED Overview
- 2. Core Concepts
 - 2.1 I2C Bus
 - 2.2 SSD1306 Display Principle
 - 2.3 Font Rendering Principle
 - 2.4 Core API
- 3. Hardware Dependencies
 - 3.1 Pin Mapping
 - 3.2 OLED Parameters
- 4. Project Architecture and Flow
 - 4.1 File Architecture
 - 4.2 Running Flow
- 5. Source Code Details
 - 5.1 Entry main.py — Font Selection
 - 5.2 SSD1306 Driver (ssd1306.py)
 - 5.3 Font Data (fonts.py)
- 6. Operation Procedure
 - 6.1 File Upload
 - 6.2 Serial Output Example
 - 6.3 Expected Behavior
 - 6.4 Verification Method
- 7. Common Issues

1. I2C OLED Overview

SSD1306 is a monochrome OLED driver chip with 128×32 resolution, communicating with the host through the **I2C bus** (only the two lines SDA / SCL are needed). This experiment is based on a pure MicroPython self-written SSD1306 driver (no third-party dependencies), displaying "HI YAHBOOM" on the OLED with an 11×18 pixel font.

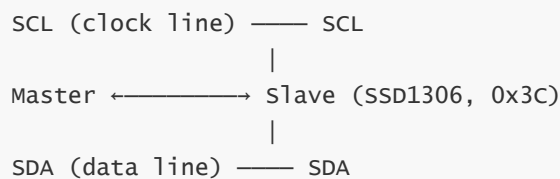
Typical application scenarios:

Application	Example
Data display	Temperature/humidity, time, voltage, etc.
Status panel	Device running status, IP address
Menu interface	Embedded UI, parameter settings
Debug output	Offline log display

2. Core Concepts

2.1 I2C Bus

I2C is a two-wire serial bus invented by Philips:



- **SCL**: The master generates the clock to control the transmission rhythm
- **SDA**: Bidirectional data line, address + data + acknowledge

I2C requires two wires: SCL (clock) and SDA (data), both with the clock generated by the host. Each slave has a 7-bit address; the host sends the address first after START to select the target slave.

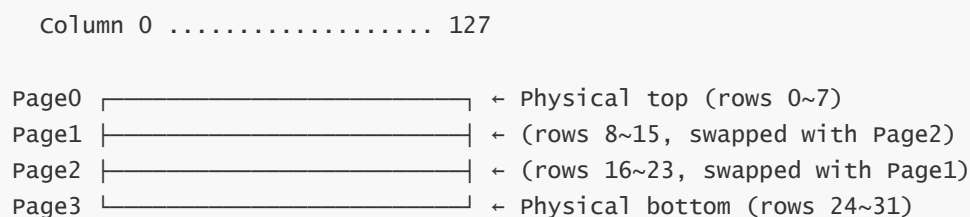
```
Master write: START → Device address(7bit)+W(1bit) → ACK(1bit) → Register
address(8bit) → ACK(1bit) → Data(8bit) → ACK(1bit) → STOP
Master read: START → Device address(7bit)+W(1bit) → ACK(1bit) → Register
address(8bit) → ACK(1bit) →
                RESTART → Device address(7bit)+R(1bit) → ACK(1bit) → Data(8bit) →
                NAK(1bit) → STOP
```

In this experiment the I2C address is `0x3C` (SSD1306 default), SCL frequency 100kHz (standard mode).

Each slave has a unique 7-bit address (SSD1306 default `0x3C`).

2.2 SSD1306 Display Principle

The SSD1306 128×32 display area is divided into **4 pages**, each page being 8 rows (128 columns × 8 rows/page):

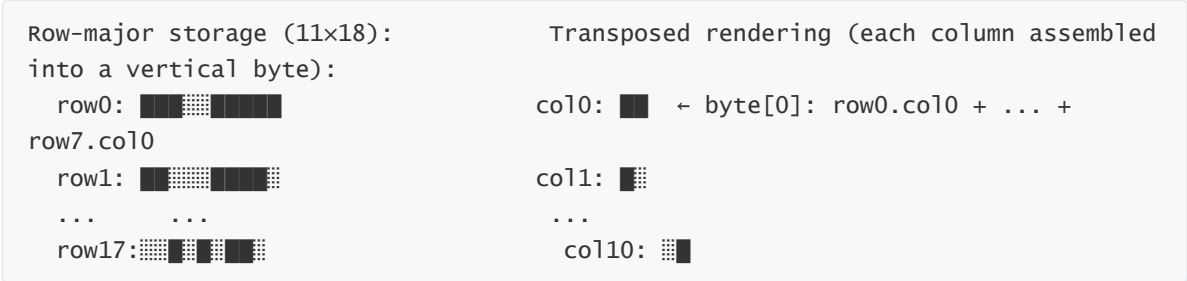


Key: The OLED traces on this board cause a physical swap between Page1 ↔ Page2, which is handled in the code.

Each column of 8 vertical pixels is represented by one byte: bit 0 = the bottommost pixel of that column, bit 7 = the topmost.

2.3 Font Rendering Principle

The glyph data uses **row-major order**: each data entry corresponds to a row of pixels. During rendering it is **transposed** column by column — taking the bits of the same column from each row, assembling them into a vertical 8-pixel byte, and writing them to the corresponding SSD1306 page.



This experiment provides **3 fonts**: 7×10, 11×18, 16×26. Modify the `OLED_FONT` variable to switch, the default is `'11x18'`.

2.4 Core API

```
from machine import Pin, I2C

# I2C initialization
i2c = I2C(0, scl=Pin(42), sda=Pin(41), freq=100000)

# SSD1306 operations
from I2C_OLED import ssd1306
oled = ssd1306.SSD1306(i2c, addr=0x3C)
oled.show_str(0, "HI YAHBOOM")           # Default 11x18 font
oled.show_str(0, "HELLO", font='16x26')  # Switch to 16x26 font
oled.show_str(0, "OK", font='7x10')       # Switch to 7x10 font
oled.clear()                             # Clear screen
```

3. Hardware Dependencies

3.1 Pin Mapping

GPIO	Peripheral	Mode	Description
41	I2C0 SDA	I2C	I2C0 data pin
42	I2C0 SCL	I2C	I2C0 clock pin

3.2 OLED Parameters

Parameter	Value	Description
Driver chip	SSD1306	Monochrome OLED driver IC
Resolution	128×32	128 columns × 32 rows
I2C address	0x3C	7-bit address (SA0=0)
I2C frequency	100kHz	Standard mode (SSD1306 supports up to 400kHz)
Interface	I2C	Only requires the two lines SDA + SCL

4. Project Architecture and Flow

4.1 File Architecture

```
2. I2C_OLED/  
├─ main.py           # Project entry (initialize I2C → SSD1306 → display)  
├─ ssd1306.py        # SSD1306 driver layer (I2C commands → page/column  
addressing → transposed rendering)  
└─ fonts.py          # Font data layer (5 ASCII glyph bitmaps)
```

Three-layer architecture:

```
main.py (application layer)  
  ↓  
ssd1306.py (driver layer) – transposed rendering, page addressing, I2C TX/RX  
  ↓  
fonts.py (data layer) – character bitmap data  
  ↓  
machine.I2C (hardware abstraction) – I2C bus
```

4.2 Running Flow

```
1. Upload ssd1306.py and fonts.py to ESP32-S3 (I2C_OLED directory)  
2. Upload main.py, modify OLED_FONT to select the font, save and run  
   └─ I2C(0, scl=42, sda=41, freq=100kHz) // Initialize I2C  
   └─ SSD1306(i2c, addr=0x3C)             // Initialize OLED  
     │ └─ Send the initialization sequence (about 15 commands)  
     │ └─ clear() clear screen  
     └─ Turn on the display  
   └─ oled.show_str(0, "HI YAHBOOM", font=OLED_FONT) // Render with the selected  
font
```

5. Source Code Details

Complete source code: [5.Source Code](#) → [MicroPython](#) → [3.Serial Communication and Storage System](#) → [2. I2C_OLED](#)

5.1 Entry main.py — Font Selection

The `font` parameter of `show_str(x, s, font)` can be `'7x10'` / `'11x18'` / `'16x26'`; if not passed, it defaults to `'11x18'`. Modify the `OLED_FONT` variable to switch fonts.

```
from machine import Pin, I2C          # Pin and I2C classes  
from I2C_OLED import ssd1306, fonts  # SSD1306 driver + fonts  
  
# ---- Hardware configuration ----  
SDA_PIN = 41          # I2C data pin  
SCL_PIN = 42          # I2C clock pin  
OLED_ADDR = 0x3C      # SSD1306 7-bit address  
  
# ---- Initialize ----  
print('Init I2C...')  
i2c = I2C(0, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=100000)  
  
print('Init SSD1306...')  
oled = ssd1306.SSD1306(i2c, addr=OLED_ADDR)
```

```
# ---- Font selection ----
# show_str(x, s, font) - font: '7x10' | '11x18'(default) | '16x26'
OLED_FONT = '11x18' # Change this to switch font

oled.show_str(0, 'HI YAHBOOM', font=OLED_FONT)
print('Font: %s' % OLED_FONT)
```

5.2 SSD1306 Driver (ssd1306.py)

I2C low-level TX/RX — command bytes are prefixed with `0x00`, data bytes with `0x40`:

```
class SSD1306:
    def __init__(self, i2c, addr=0x3C):
        self.i2c = i2c          # I2C bus object
        self.addr = addr        # OLED I2C address
        self._init_seq()        # Run the init sequence

    def _cmd(self, c):
        self.i2c.writeto(self.addr, b'\x00' + bytes([c])) # Command: prefix
0x00

    def _data(self, d):
        self.i2c.writeto(self.addr, b'\x40' + bytes([d])) # Data: prefix 0x40
```

Initialization sequence — sends about 15 commands after power-up to configure the chip:

```
def _init_seq(self):
    import time
    time.sleep_ms(100)          # Power stable

    self._cmd(0xAE)             # Display OFF
    self._cmd(0xD5); self._cmd(0x80) # Clock
    self._cmd(0xA8); self._cmd(0x1F) # MUX=32 (128x32)
    self._cmd(0x8D); self._cmd(0x14) # Charge pump ON
    self._cmd(0x20); self._cmd(0x02) # Page addressing mode
    self._cmd(0xA1)              # Segment remap (left/right flip)
    self._cmd(0xC0)              # COM scan normal
    self._cmd(0xDA); self._cmd(0x12) # COM pins configuration
    self._cmd(0x81); self._cmd(0xCF) # Contrast
    self._cmd(0xA4)              # Entire display ON
    self._cmd(0xA6)              # Normal display (no inversion)
    self.clear()                 # Fill the entire screen with 0
    self._cmd(0xAF)              # Display ON
    time.sleep_ms(100)
```

Page positioning — SSD1306 128×32 = 4 pages × 128 columns; set the page number + column address before writing data:

```

def set_pos(self, page, col):
    self._cmd(0xB0 + page)      # Page base + page num
    self._cmd(col & 0x0F)      # Column low nibble
    self._cmd(0x10 | (col >> 4)) # Column high nibble

def clear(self):
    for p in range(4):
        self.set_pos(p, 0)
        for _ in range(128):
            self._data(0x00)

```

Transposed rendering — the glyph data is row-major (each entry = horizontal bits of one row of pixels), but SSD1306 needs vertical bytes. Traverse column by column, take the bits of the same column from each row, and assemble them into one vertical 8-pixel byte:

```

def _render_page(self, x, data, w, sr, n, pg):
    self.set_pos(pg, x)
    for col in range(w):          # Per column
        b = 0
        for r in range(n):        # Per row
            if _glyph_bit(data, sr + r, col):
                b |= (1 << (n - 1 - r)) # Row bit → vertical byte (top row
= high bit)
        self._data(b)             # Write to this column

```

Character drawing — taking the 11×18 font as an example. Since the OLED traces on this board cause a **physical swap between Page1↔Page2**, the rendering deliberately writes rows 0~7 to Page2 and rows 8~15 to Page1:

```

def show_11x18(self, x, ch):
    d = fonts.Font_11x18_data
    i = (ord(ch) - ord(' ')) * 18      # 18 entries per character, indexed
from space
    self._render_page(x, d[i:], 11, 0, 8, 2) # rows 0-7 → Page2 (swapped)
    self._render_page(x, d[i:], 11, 8, 8, 1) # rows 8-15 → Page1 (swapped)
    self._render_page(x, d[i:], 11, 16, 2, 3) # rows 16-17 → Page3

```

String interface — the `font` parameter switches fonts; if not passed, it defaults to 11x18:

```

def show_str(self, x, s, font='11x18'):
    draw = {
        '7x10': (self.show_7x10, 7),
        '11x18': (self.show_11x18, 11),
        '16x26': (self.show_16x26, 16),
    }.get(font, (self.show_11x18, 11))

    for ch in s:
        draw[0](x, ch)          # Call the character drawing
function for the corresponding font
        x += draw[1]           # Step by the character width

```

5.3 Font Data (fonts.py)

ASCII lookup tables for the 3 fonts (space to ~), stored as arrays. Each entry is 16-bit; character index = $(\text{ord}(\text{ch}) - \text{ord}(' ')) \times \text{number of entries}$:

Font	Width×Height (px)	Entries/Character	Format
Font_7x10	7×10	10	Row-major
Font_11x18	11×18	18	Row-major
Font_16x26	16×26	26	Row-major

```
# 3 ASCII font bitmap data
Font_7x10_data = [
    0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,
    0x0000,0x0000,0x1000,0x1000, ... # 10 rows of pixel data for '!'
]

Font_11x18_data = [
    0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,
    0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,0x0000,
    0x0000,0x0000,0x0000,0x0C00,0x0C00, ... # 18 rows of pixel data for '!'
]

# Font_16x26_data - 26 entries/character, row-major
```

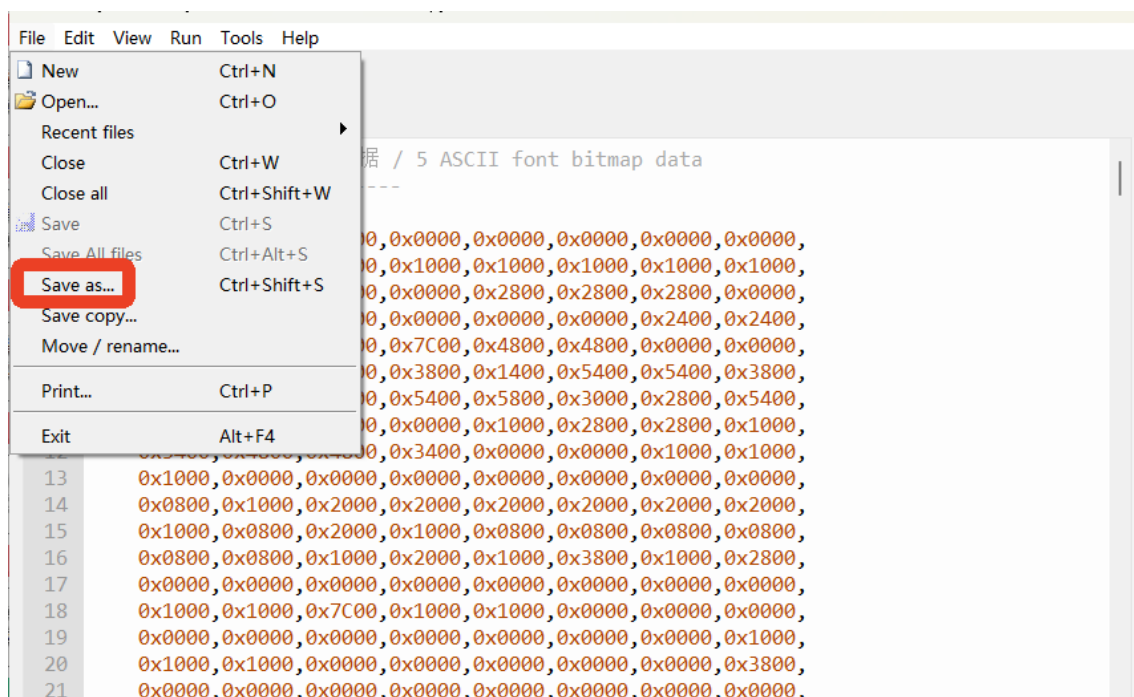
6. Operation Procedure

6.1 File Upload

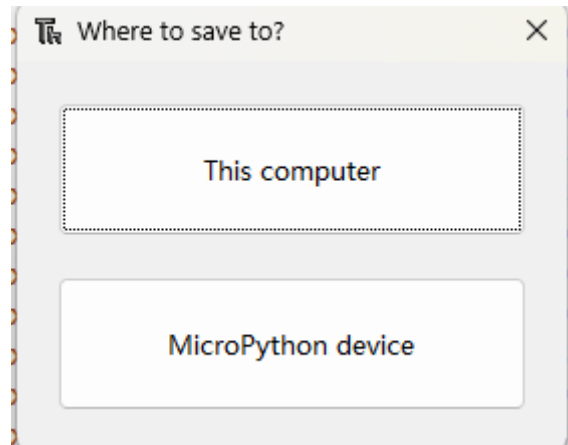
Thonny IDE operation steps: See [MicroPython → 1. Before Development → 3. Thonny IDE](#).

1. In Thonny IDE, upload `ssd1306.py` and `fonts.py` to ESP32-S3

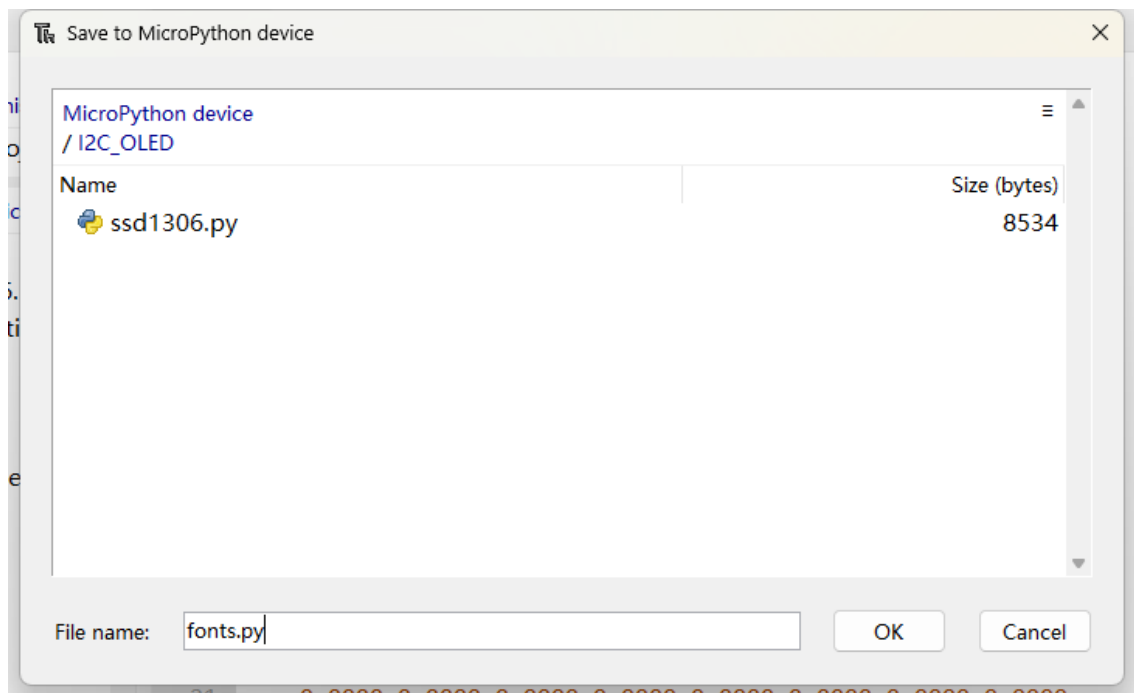
① Select the file → Save as...



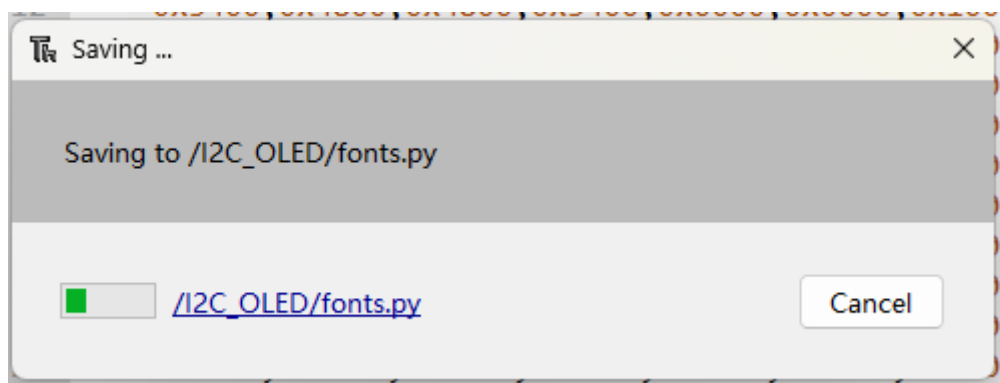
②Select MicroPython device



③Name the file or create a new folder (name the file in the new folder)

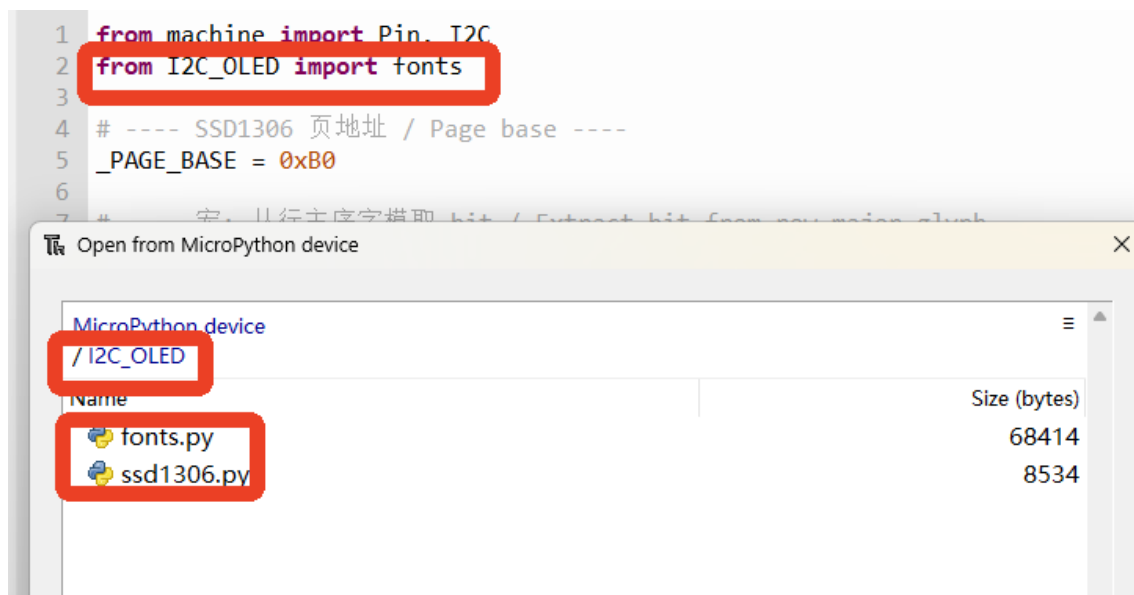


④Wait for the cache



2. Create the `I2C_OLED` directory on the device and move the two files into it

Note: The library name and the name of the newly created folder must correspond, and the names of the .py files placed inside must also correspond one by one.



3. Run `main.py`

6.2 Serial Output Example

```
Init I2C...
Init SSD1306...
Font: 11x18
```

```
Init I2C...
Init SSD1306...
Font: 11x18
```

6.3 Expected Behavior

The OLED screen displays "HI YAHBOOM". Modify the `OLED_FONT` variable (e.g. `'7x10'` / `'16x26'`, etc.) to switch the font size.



6.4 Verification Method

Action	Expected result
Run main.py	OLED displays "HI YAHBOOM"
Change "HI YAHBOOM" to "HELLO"	Display changes to "HELLO"
Change <code>OLED_ADDR=0x3D</code>	OLED does not display (wrong address)
Run <code>oled.clear()</code> first	Clears the screen, then displays
<code>OLED_FONT = '16x26'</code>	Font becomes larger; 16×26 can only display 8 characters
<code>OLED_FONT = '7x10'</code>	Font becomes smaller; 7×10 can display 18 characters per line

7. Common Issues

Symptom	Cause	Solution
OLED does not light up	Wrong I2C address	The I2C address is usually <code>0x3C</code> (some modules are <code>0x3D</code>); check the module silkscreen
Garbled characters or abnormal display	Page mapping error	Confirm the Page1/Page2 swap logic in <code>ssd1306.py</code> matches the hardware
<code>ImportError: no module named 'I2C_OLED'</code>	Files not uploaded or directory missing	Create the <code>I2C_OLED</code> directory on the device and put <code>ssd1306.py</code> and <code>fonts.py</code> into it
Screen jitter or flicker	I2C frequency too high	Lower <code>freq=</code> to 50000 or 100000
I2C cannot scan the device	Wiring error or not powered	Use a multimeter to check SDA/SCL continuity and confirm the OLED is powered properly